

Deployment, observability and capstone

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you put Project 1 in front of the agent as a cheap pre-filter and measure cost per resolved ticket before and after. Then you promote an approved artifact to the live target behind a canary, release images through the delivery pipeline with a human approval, and read the alarm that takes a bad release back.

The EU AI Act applies now; the Omnibus deferred the high-risk dates

Date	What applies
2024-08-01	the Act enters into force
2025-02-02	prohibited practices, Article 5
2025-08-02	general-purpose AI provider obligations
2026-07-27	the Digital Omnibus on AI enters into force
2026-08-02	general application, with the Article 50 transparency rules
2027-12-02	high-risk in Annex III areas: biometrics, critical infrastructure, education, employment, credit, migration
2028-08-02	high-risk AI embedded in regulated products, Annex I

Three frameworks give governance a shape an auditor recognises

NIST AI RMF 1.0

Four functions: **Govern, Map, Measure, Manage**. Released 2023-01-26; NIST says a revision is under way.

NIST AI 600-1

The Generative AI Profile, 2024-07-26: **12 risks** GenAI creates or worsens, among them confabulation, data privacy, information security and value chain integration.

ISO/IEC 42001:2023

The first international **management system** standard for AI, and certifiable: AWS announced certification in 2024-11 for Bedrock and three other services.

The course already produces the evidence: the use case catalog maps, the harness and tiered gates measure, the kill switch and approvals manage.

AI TRiSM is a checklist: every control in this course sits in one of four layers

AI governance

Use case catalog, agent registry, model and prompt registries, calibrated Judge, tiered gates.

Runtime inspection and enforcement

Guardrails, the Cedar policy, the approval gate, drift, kill switch, spend caps.

Information governance

Redaction, audience and currency filters, data classes per use case.

Infrastructure and stack

Least privilege per tenant, secrets by reference, KMS, scanning, SBOM, signing.

<https://www.gartner.com/en/articles/ai-trust-and-ai-risk>

Sector rules sit on top: US bank model risk guidance changed in April 2026

SR 26-2

Federal Reserve, OCC and FDIC, 2026-04-17. It supersedes SR 11-7, the 2011 guidance most model risk teams still quote.

- It keeps the core principles and asks each bank to scale model risk management to its own risk profile
- US credit denials need accurate, specific reasons: "there is no special exemption for artificial intelligence" (CFPB, 2023)
- In the EU, credit scoring is Annex III high-risk; fraud detection is explicitly excluded

Cost is a design input, and cost per resolution is the number that matters

98%

of FinOps practitioners now manage AI spend: up from 63 percent in 2025 and 31 percent in 2024 (State of FinOps 2026)

- **The price falls:** reaching a fixed capability has got about 47 percent cheaper per quarter since 2023, and agents spend far more tokens per task
- **Gartner predicts** agentic AI will resolve 80 percent of common service issues by 2029, and that GenAI cost per resolution will exceed 3 USD by 2030, more than many offshore human agents

Value follows the operating model more than the model

10 / 20 / 70

where BCG's AI leaders put their effort: algorithms, data and technology, then people, processes and culture

Google MLOps level	Where this course reaches it
0, manual process	the notebooks
1, pipeline automation	Projects 1 and 2
2, CI/CD of the pipeline	Projects 3, 4 and the capstone

A team with the whole platform that still promotes by hand is at level 0.

The ladder rewards scope, so choose your next depth on purpose

- One public example, Dropbox: IC1 to IC4, then **Staff, Principal, Senior Principal**; managers M3 to M7
- ML engineers share the software ladder; levels differ by "scope, collaborative reach, and levers for impact"
- AI engineer topped LinkedIn's 2026 US Jobs on the Rise; top feeders: software engineer, data scientist

Growing toward	Learn next
Senior AI engineer	evaluation design, cost modelling, one serving stack in depth
Agentic AI engineer	tool design, agent evaluation, MCP and A2A security
MLOps and platform	multi-tenant gateways, delivery, SLOs
AI safety and governance	NIST AI RMF, ISO/IEC 42001, the EU AI Act in practice

The system runs; now the organisation asks who approved it and what it costs

PART 1

Foundations and the AI service layer

PART 2

Machine learning in production

PART 3

Deep learning and representation

PART 4

Generative AI: retrieval and evaluation

PART 5

Agentic AI systems

PART 6

Deployment, observability and capstone

Four services and an agent run on the platform, each registered and behind the model gateway. Nothing in this part deploys from your laptop except through the platform's own commands: the registry, the delivery pipeline, the canary.

Today you promote, deliver and price the whole system

- 1 Where you stand on the platform, and what it costs
- 2 Route cheap first
- 3 Cost per resolved ticket, before and after
- 4 The promotion drill: approve, go live behind a canary
- 5 Images through the delivery pipeline, and the alarm that rolls back
- 6 Dashboards, SLOs and the exported metrics
- 7 Lower and higher environments
- 8 Session check, demos, debrief

One file holds the numbers

`sessions/session-06/NOTES.md` : three cost sheet figures, two costs per resolution and the gateway spend, the promotion, the rollback command, one SLO row.

The session check reads it and asks for what is missing.

Who deploys, who promotes, who pays depends on the mode

	Cohort mode	Solo mode	Local track
Platform	the instructor, once per track	you, in Part 0	<code>make local-up</code>
You are	a tenant, your handle on every resource	the tenant <code>solo</code>	the tenant in <code>.env</code>
Live target	shared; the instructor promotes a volunteer's version	yours	yours, plus <code>higher</code>
Stop and destroy	the instructor	you, before you leave	<code>make local-down</code>
Cost	credit to request: 15 USD per learner	AWS about 56 USD, Google Cloud about 20 USD	16 GB memory, about 40 GB disk

Route cheap first: Project 1 scores a ticket for nothing

```
triage = registry.execute("classify_urgency", {"subject": subject, "body": body})
if priority == "P0" and p0 >= policy.p0_confidence:    -> proposed escalation, no model call
if priority in policy.economy_priorities:              -> run_agent(role=ECONOMY, small caps)
else:                                                  -> run_agent(role=WORKHORSE)
```

Confident P0

A proposal to the duty manager, cost zero. The approval gate still applies.

Economy

Nova Micro on AWS; gpt-oss-120b with Economy caps on Google Cloud; gpt-oss-20b on Local.

Everything else

The Workhorse loop from the agent part, screener included.

The capstone is two numbers on the board, and the gateway's bill beside them

- 1 Read your key's `spend` on the gateway
- 2 The same twenty tickets through the plain loop and through your router
- 3 Resolved count, cost per resolved ticket, mean steps, the router's branches
- 4 Read `spend` again; explain any gap with the meter

Write yours, not a slide's

The course team's Project 1 artifact split the first twenty as 16 Workhorse, 2 Economy, 2 P0 direct. Your split depends on your model.

Dollar figures on the delivery week's model come from the room.

Promotion is an approval, a stage change with a reason, and a canary that rolls back without a person

	Approval	Live target and canary	Rolls back on
AWS	<code>APPROVED</code> event deploys your serverless endpoint	<code>Stage.LIVE</code> into <code>northwind-live-triage</code> , SageMaker blue/green, 10 percent for 5 minutes	the <code>-5xx</code> , <code>-p95</code> , <code>-drift</code> alarms
Google Cloud	alias moves; your Cloud Run service rolls a revision	endpoint <code>northwind-live-triage</code> , traffic split at 10, then 0	a person, paged by alert policies
Local	alias <code>approved</code> in MLflow	<code>serving-canary</code> at weight 10, then <code>live</code>	a person, reading Grafana

Code promotes through the delivery pipeline: built once, pushed by digest, approved by a person

	Starts the release	Human approval	Canary
AWS	a push to <code>main</code> : CodePipeline <code>northwind-delivery</code> via CodeConnections	the <code>Approve</code> stage	policy alias via CodeDeploy, 10 percent for 15 minutes; live resolver on the new digest
Google Cloud	<code>make release-gcp</code> or the <code>northwind-main</code> trigger	<code>make approve-gcp</code> on the rollout	Cloud Deploy canary on Cloud Run at <code>canary_percent</code>
Local	<code>make local-up</code>	you	<code>make local-canary WEIGHT=10</code>

Pull requests stay in GitHub Actions. Actions never pushes an image to a cloud registry.

A canary nobody can stop is a slower outage

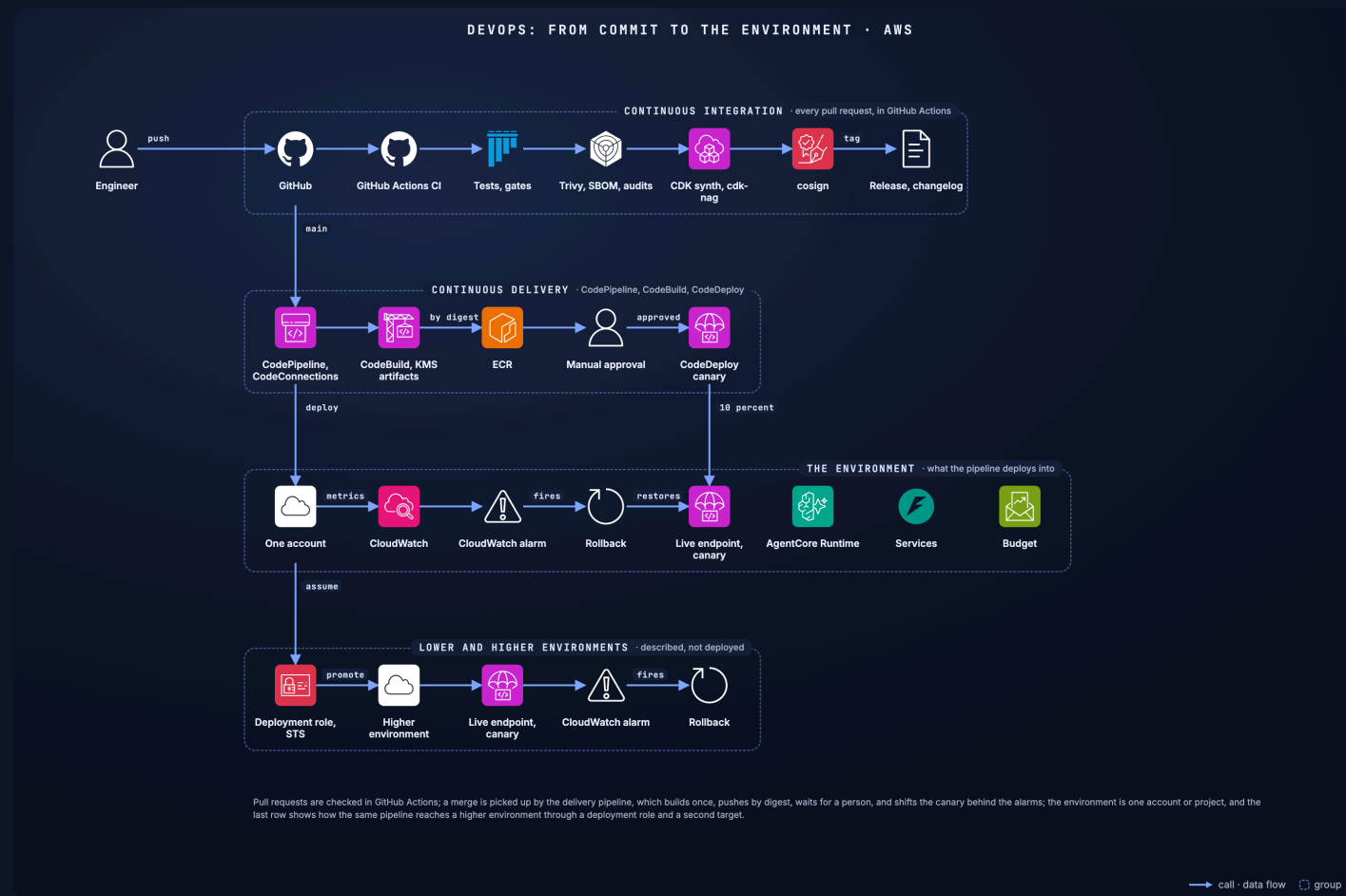
- **AWS:** both canaries roll back on their own. The errors alarm needs more than five errors in each of two five-minute periods, so the alias canary holds 15 minutes
- **Google Cloud:** alert policies page a person; `-to-tags stable=100` or abandon the rollout
- **Local:** nothing moves traffic; `make local-canary WEIGHT=0` by hand

Not measured yet

The time from a broken canary to the rollback has not been measured on this platform. Until the validation run, the honest sentence is "within the alarm's evaluation window".

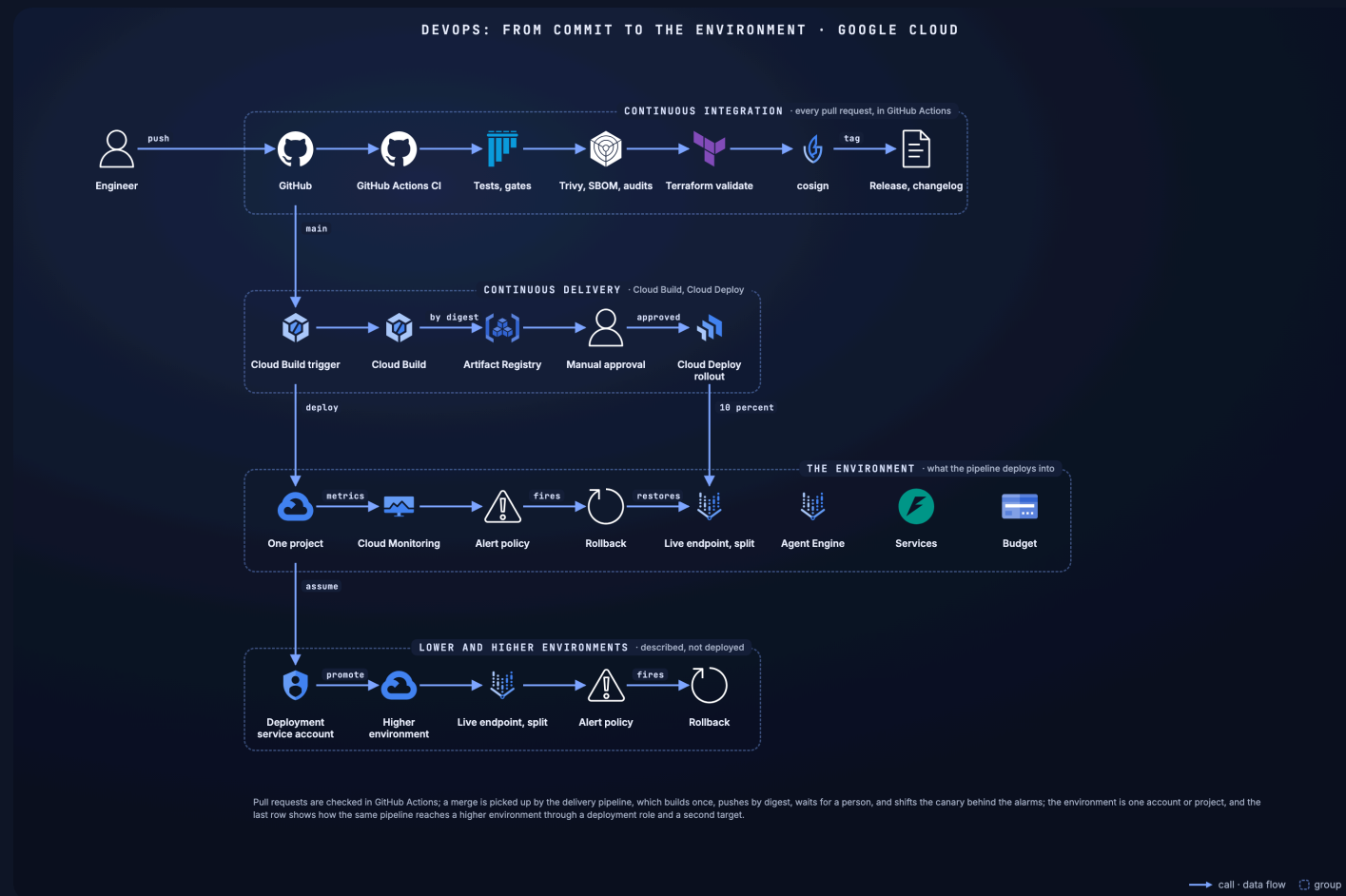
Write your track's rollback command in `NOTES.md` : that is the runbook.

DevOps, AWS track



Continuous integration in GitHub Actions; delivery through CodePipeline, CodeBuild, ECR by digest, a manual approval and the CodeDeploy canary; the one environment with the alarm that restores; below, the higher environment reached through a deployment role, described and not deployed.

DevOps, Google Cloud track



The same pull-request checks; Cloud Build, Artifact Registry and a Cloud Deploy pipeline with one target and an approval, a Cloud Run canary under the alert policies. On the Local track (`devops-local.png`) the stages are compose profiles driven by make, with the `higher` tier running for real.

AI systems degrade silently; three panels make it loud

A model answers slower

Nothing errors. Users wait.

p95 latency

A retriever returns nothing

The model refuses, or answers from nothing.

Refusal rate

A judge drifts

The harness passes what it should fail.

Cost per resolution

AWS: the CloudWatch dashboard, metrics written as Embedded Metric Format lines, traces in X-Ray. Google Cloud: the `northwind` dashboard from log-based metrics, Cloud Trace. Local: Grafana on :3000, Prometheus, Jaeger, Evidently.

While the error budget is spent, nothing promotes

SLI	SLO	Measured by
p95 latency, policy	8 seconds	the latency alarms and the exported p95
p95 latency, triage and semantic	2 seconds	the exported p95
Drift	zero open <code>drift_alert</code> lines	the metric filter or log-based metric per service
Cost per resolution	at most 1.5 times the agent baseline	the gate's system tier, the spend cap, the budget
Rollout safety	no rollout reaches 100 percent while an alarm is in ALARM	the canary's alarms

A higher environment is the same code, a second target and one role that may cross

A second target

A Deploy action per environment in CodePipeline; a second Cloud Deploy target; the `higher` profile locally. Each with its own approval and canary.

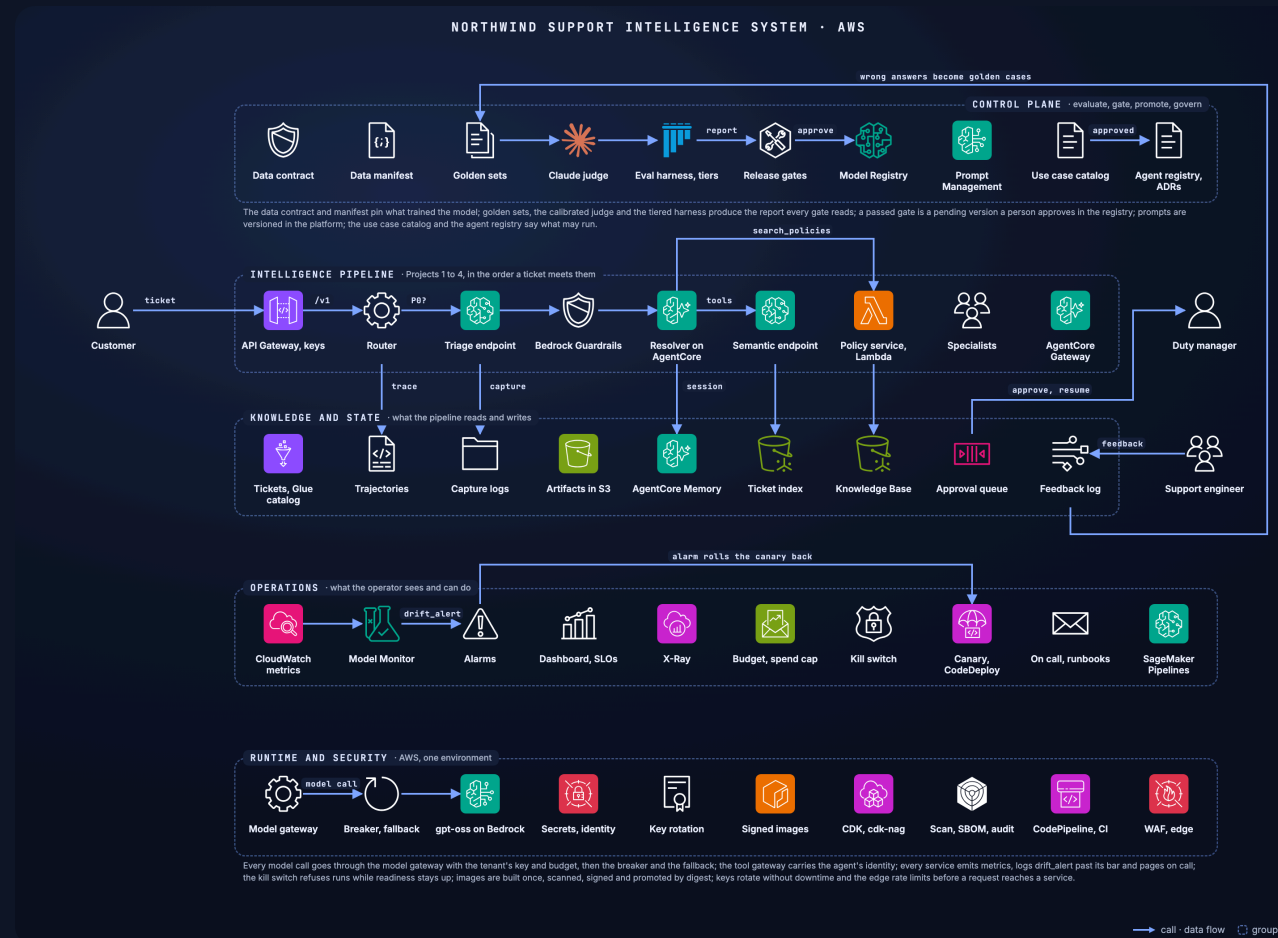
A deployment role

`northwind-<env>-deployer` or a deployer service account that trusts exactly one principal: the pipeline's role below.

Promotion by reference

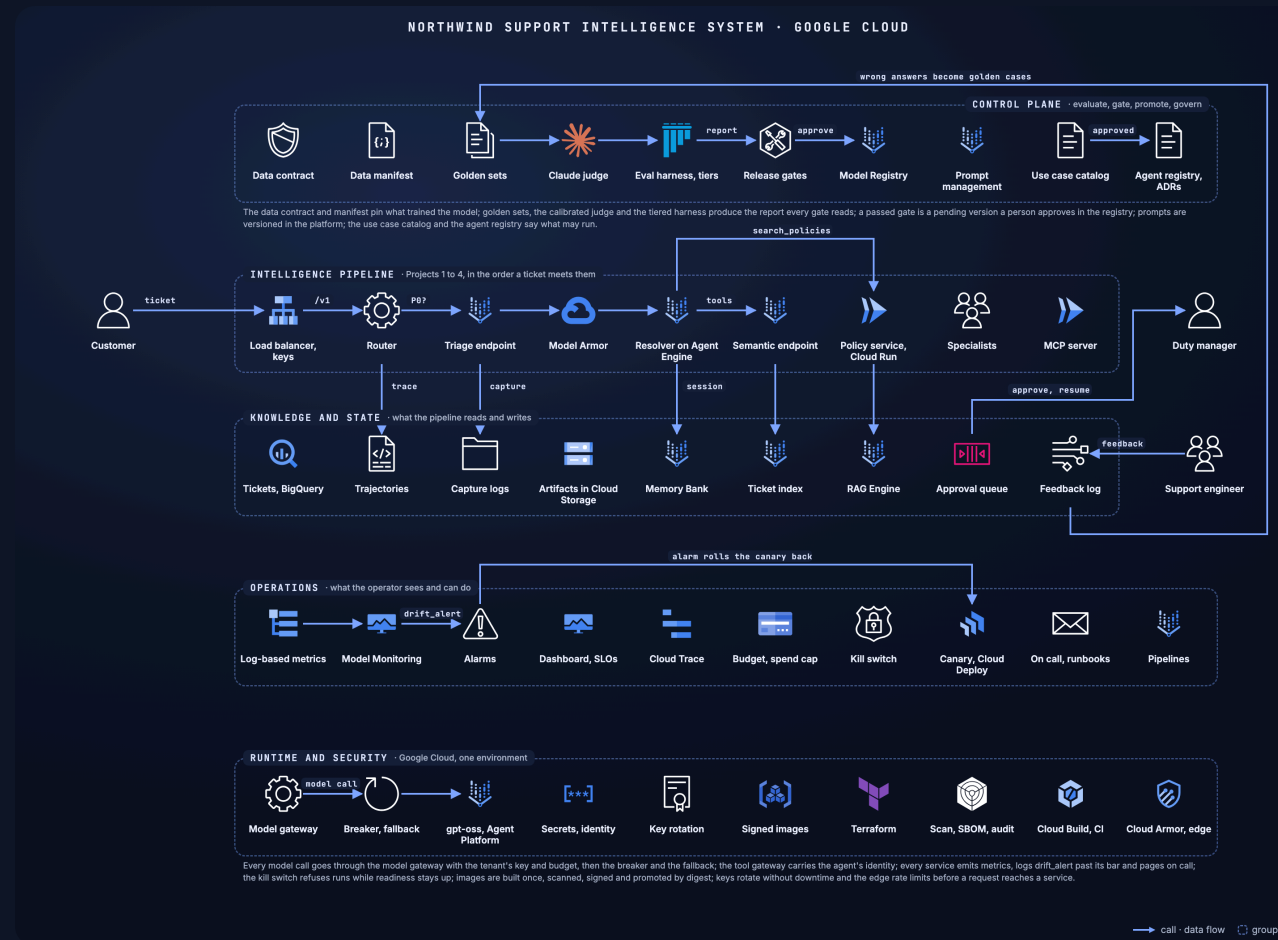
Images by digest, model versions copied with the same artifact URI. Never a rebuild, never a retrain.

The whole system, AWS track



Control plane, the intelligence pipeline in ticket order, knowledge and state, operations, runtime and security. Two loops people close: approvals and feedback.

The whole system, Google Cloud track



The same rows on Cloud Run, Agent Engine, the Agent Platform (formerly Vertex AI), Cloud Trace and log-based metrics. The Local track's figure ([system-local.png](#)) keeps every row with open-source services.

Acceptance criteria

1. The router's three branches pass; a confident P0 costs nothing; its card lists both roles
2. Two costs per resolved ticket and the gateway spend in `NOTES.md`
3. An approved version live behind a canary, or watched in cohort mode, with the reason
4. Your track's release path and rollback command written down
5. One SLO row with its panel; the path to a higher environment drawn

47 then 24

`make session06` : the capstone tests, then the AWS synth review when `deploy/aws/.venv` exists

Questions worth asking

- Of every failure across the six parts, which would have reached production in your setup, and which TRiSM layer would have caught it?
- Your router saves money on P0 and Economy tickets. What happens when Project 1's P0 recall drops, and which alarm tells you first?
- Which identity in your organisation can write to production without going through a pipeline?
- If the model vendor changed tomorrow, what in this repo would you touch?